

// Jivops

Caddy

Guía completa — 20 lecciones

Fundamentos · Reverse proxy · TLS y seguridad · Operación y producción

Contenido

Módulo 1: Fundamentos

- Lección 01 — Qué es Caddy y su HTTPS automático
- Lección 02 — Caddyfile y bloques de sitio
- Lección 03 — Matchers y placeholders
- Lección 04 — file_server, encode y sitios estáticos
- Lección 05 — Comandos y servicio

Módulo 2: Reverse proxy

- Lección 06 — reverse_proxy
- Lección 07 — Balanceo de carga
- Lección 08 — Health checks activos y pasivos
- Lección 09 — Cabeceras y WebSockets
- Lección 10 — handle, route y orden de directivas

Módulo 3: TLS y seguridad

- Lección 11 — Cómo funciona el HTTPS automático
- Lección 12 — Certificados: DNS challenge, wildcard y tls internal
- Lección 13 — TLS bajo demanda
- Lección 14 — Autenticación y cabeceras de seguridad
- Lección 15 — Restricción por IP y límites

Módulo 4: Operación y producción

- Lección 16 — API de administración y configuración JSON
- Lección 17 — Módulos y xcaddy
- Lección 18 — Logs y métricas
- Lección 19 — Docker, Kubernetes y almacenamiento de certificados
- Lección 20 — Proyecto final: Caddy listo para producción

Módulo 1: Fundamentos

Lección 01 — Qué es Caddy y su HTTPS automático

Un servidor web y proxy inverso escrito en Go que obtiene, instala y renueva certificados TLS automáticamente (HTTPS por defecto con Let's Encrypt o ZeroSSL), usa una configuración muy sencilla (Caddyfile) y soporta HTTP/2 y HTTP/3.

Obtiene y renueva certificados solo, sin pasos manuales ni caducidades olvidadas, y activa HTTPS por defecto.

Lección 02 — Caddyfile y bloques de sitio

Con bloques de sitio que empiezan por la dirección (por ejemplo ejemplo.com) y contienen directivas entre llaves; un bloque global opcional al principio fija opciones generales, y los snippets con import permiten reutilizar fragmentos.

Para opciones que afectan a todo Caddy, como el correo para ACME, la configuración del servidor o el registro.

Lección 03 — Matchers y placeholders

Los matchers (como `@api path /api/*`) seleccionan las peticiones a las que se aplica una directiva según ruta, cabecera, método o IP, y los placeholders (como `{http.request.host}`) insertan valores de la petición o del entorno en la configuración.

Defines un matcher con nombre, por ejemplo `@api path /api/*`, y lo usas en la directiva (`reverse_proxy @api ...`).

Lección 04 — file_server, encode y sitios estáticos

Con `root * /ruta` para el directorio, `file_server` para servir los ficheros y `encode gzip zstd` para comprimir; `try_files` permite usar el `index.html` en aplicaciones de una sola página.

Para servir `index.html` cuando la ruta no corresponde a un fichero, de modo que el enrutado lo resuelva el frontend.

Lección 05 — Comandos y servicio

`caddy run` (primer plano), `caddy start` (segundo plano), `caddy validate` (comprobar la configuración), `caddy fmt` (formatear el Caddyfile), `caddy adapt` (convertir a JSON) y `caddy reload` (recarga sin cortes); en producción se ejecuta como servicio `systemd`.

Porque aplica la nueva configuración sin cortar las conexiones activas, y si es inválida mantiene la anterior.

Módulo 2: Reverse proxy

Lección 06 — reverse_proxy

Con la directiva `reverse_proxy` seguida de uno o varios destinos (`reverse_proxy 127.0.0.1:8080`); Caddy añade por defecto las cabeceras `X-Forwarded-*` y gestiona HTTP/2, WebSockets y streaming sin configuración extra.

Las cabeceras `X-Forwarded-For`, `X-Forwarded-Proto` y `X-Forwarded-Host` con los datos de la petición original.

Lección 07 — Balanceo de carga

Indicando varios destinos en `reverse_proxy` y eligiendo `lb_policy` (`random` por defecto, `round_robin`, `least_conn`, `ip_hash`, etc.), con reintentos hacia otros destinos si uno falla (`lb_try_duration`).

Con conexiones largas o de duración desigual, para enviar al destino con menos conexiones activas.

Lección 08 — Health checks activos y pasivos

Activas (`health_uri`, `health_interval`: sondea una URL periódicamente) y pasivas (`fail_duration` y `max_fails`: marca un destino como no sano tras fallos reales); un destino no sano deja de recibir tráfico.

La activa detecta caídas aunque no haya tráfico y la pasiva detecta fallos reales bajo carga.

Lección 09 — Cabeceras y WebSockets

Con `header_up` y `header_down` dentro de `reverse_proxy` para las cabeceras hacia el backend y hacia el cliente, y con la directiva `header` para respuestas; los WebSockets funcionan sin configuración especial.

Configurando `trusted_proxies` para confiar en la cabecera `X-Forwarded-For` del proxy anterior y evitar suplantación.

Lección 10 — handle, route y orden de directivas

`handle` agrupa directivas y es excluyente entre bloques `handle` hermanos (se aplica el primero que coincide), manteniendo el orden predefinido de directivas, mientras que `route` respeta el orden literal en que se escriben, útil cuando importa la secuencia.

Porque las directivas se ejecutan en un orden determinado y cambiarlo puede producir resultados distintos (por ejemplo, una redirección antes o después de una reescritura).

Módulo 3: TLS y seguridad

Lección 11 — Cómo funciona el HTTPS automático

Que el dominio apunte por DNS al servidor, que los puertos 80 y 443 sean accesibles desde internet para los desafíos ACME, que el nombre de sitio sea un dominio público y que el directorio de datos de Caddy sea persistente para conservar certificados y cuenta.

Que el DNS apunte al servidor, que los puertos 80 y 443 estén abiertos y los logs de Caddy con el motivo del fallo ACME.

Lección 12 — Certificados: DNS challenge, wildcard y tls internal

Para certificados wildcard o cuando el servidor no es accesible desde internet, validando el dominio con un registro TXT mediante un plugin del proveedor DNS; para entornos internos, tls internal usa la CA local de Caddy.

Un plugin de tu proveedor DNS compilado en Caddy y credenciales de su API para crear el registro TXT.

Lección 13 — TLS bajo demanda

Obtener el certificado en el momento de la primera conexión a un nombre desconocido, útil para plataformas con muchos dominios; sin control, cualquiera podría provocar emisiones masivas, por lo que se limita con un endpoint ask que autoriza los nombres permitidos.

Para evitar abusos que provoquen emisiones masivas y agoten los límites de la CA.

Lección 14 — Autenticación y cabeceras de seguridad

Con la directiva basic_auth y contraseñas en hash bcrypt generadas con caddy hash-password (nunca en texto plano), siempre sobre HTTPS, y cabeceras de seguridad añadidas con la directiva header.

Para que el navegador use siempre HTTPS con el sitio (HSTS).

Lección 15 — Restricción por IP y límites

Con un matcher remote_ip que permita las redes de confianza y respondiendo con error o bloqueando el resto; el límite de peticiones (rate limiting) no viene en el núcleo y se añade con un módulo.

Configurar trusted_proxies para que Caddy confíe en X-Forwarded-For y vea la IP real del cliente, no la del balanceador.

Módulo 4: Operación y producción

Lección 16 — API de administración y configuración JSON

Un endpoint HTTP (por defecto en localhost:2019) para leer y cambiar la configuración JSON en caliente, que es el formato nativo al que se convierte el Caddyfile; no debe exponerse a redes no confiables.

Porque permite cambiar la configuración sin autenticación fuerte; quien la alcance controla el servidor.

Lección 17 — Módulos y xcaddy

Compilando Caddy con los módulos deseados mediante xcaddy o descargando una compilación personalizada, porque los módulos (como proveedores DNS o rate limiting) se enlazan en el binario, no se cargan dinámicamente.

Compilar Caddy con xcaddy incluyendo el módulo del proveedor DNS, y usar ese binario.

Lección 18 — Logs y métricas

Con registros estructurados en JSON configurables con la directiva log (destino, formato y nivel) y con métricas de Prometheus habilitadas con la opción metrics, que permiten alertar sobre errores y latencia.

El aumento de errores 5xx y de latencia, y fallos de emisión o renovación de certificados.

Lección 19 — Docker, Kubernetes y almacenamiento de certificados

El directorio de datos (/data), donde se guardan certificados y la cuenta ACME, y la configuración (/config); si no se persiste, cada reinicio puede pedir certificados de nuevo y agotar los límites; en cluster, las instancias comparten el almacenamiento para coordinar la emisión.

Para coordinar la emisión y renovación, evitando que cada instancia pida los mismos certificados y se agoten los límites.

Lección 20 — Proyecto final: Caddy listo para producción

Caddyfile validado con caddy validate y recargado con reload, HTTPS automático con /data persistente, reverse_proxy con health checks y balanceo, cabeceras de seguridad, API admin protegida, logs JSON y métricas con alertas, y módulos compilados con xcaddy solo si hacen falta.

/data persistente para conservar certificados, health checks en el proxy y monitorización con logs y métricas.